

RAG systems overview

09 May 2026 01:58 PM

Simple RAG System:

It's the most basic and simple form of RAG approach anyone should know. It's the basic to know about RAG system before going in depth in RAG.

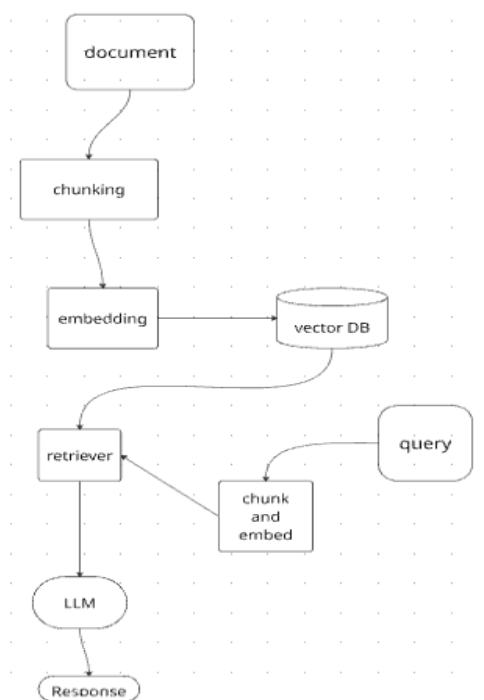
It follow the basic rule, i.e. input any doc file into it, it will retrieve all the text into chunks 1st then embed them and store in a vector space i.e. vector database.

Vector DB is the free dimensional space where all the words carrying similar terms are stored close to each other but not in the same point that's why its free space.

To read the doc, mainly I consider a PDF doc:

(* I will use langChain and LangGraph for building RAG in this series)

Work Flow:



I will assume you as an intermediate level of coder or little bit up from beginner.

The chart shows how the RAG system works clearly.

The doc is inserted and then chunked then send to embedding process, which then store those vectors in a vector DB.

When the user asks question based on the loaded doc, the query gets chunked and embedded and the retriever search the embedded chunks matches the query embedder chunks.

Then if send to the LLM and at the end we get our required response.

Sample code:

We have to load the doc (pdf) for chunking:

```

1 import fitz # PyMuPDF
2 from pathlib import Path
3
4 def load_pdf(file_name):
5     """To read the pdf and store in a file"""
6     # path = search_file(file_name)
7     doc = fitz.open(file_name)
8     text= ""
9     for page in doc:
10         text += page.get_text()
11
12     return text
13

```

We can also use the langChain pdf loader i.e.

from langchain_community.document_loaders import PyPDFDirectoryLoader

But I used PyMuPDF for loading. Cause it's okay what type of pdf loader you want to use, at the end it help to just scribe the text out of it.

Now, we have to chunk the text in this:

Here we can use 2 type of chunking approach,

1. normal manual chunking where the texts are forcefully gets cut whether the sentence is over or not, and the,
2. semantic chunkier where the chunking will happens when it hits a certain level of threshold value which keeps the sematic relation of the text.

It chunks whenever the relative meaning gets decrease or over the threshold value.

This is the comparatively the most efficient form of chunking method which is the most robust approach I should mention it.

Simple form of chunking approach:

```

1 def chunk_text(text, size=500, overlap=100):
2     """divide the texts into small chunks to feed in vector DB
3     the size is 500 by default (can be changed) and the overlap is 100 to reduce data
4
5     chunks = []
6     start = 0
7
8     while start < len(text):
9         end = start + size
10        chunks.append(text[start:end])
11        start += size - overlap
12
13    return chunks

```

Semantic chunking approach:

```

1 from sentence_transformers import SentenceTransformer
2 from sklearn.metrics.pairwise import cosine_similarity
3 import numpy as np
4
5 def semantic_chunker(text, embedding, soft_limit=350, hard_limit=550, threshold =0.75):
6     """ split text intelligently into meaningful, size-controlled segments that preserve semantic relationships,
7         making embeddings and downstream tasks (like search, summarization, or prediction) much more effective.
8         soft_limit : preferred size, tries to split earlier if the similarity drops
9         hard_limit : absolute cutoff to avoid oversized chunks
10        threshold : To make sure the sentences groped are sematically related, if the similarity is low a new chunk starts
11    """
12    chunks = []
13    current_chunk =[]
14    current_token = 0
15
16    for i, sentence in enumerate(text):
17
18        token = len(sentence.split(" "))
19        current_chunk.append(sentence)
20        current_token += token
21
22        if i > 0 and current_token > soft_limit:
23            sim = cosine_similarity([embedding[i]], [embedding[i-1]])[0][0]
24
25            if sim < threshold:
26                chunks.append(" ".join(current_chunk))
27                current_token = 0
28                current_chunk = []
29
30            if current_token > hard_limit:
31                chunks.append(" ".join(current_chunk))
32                current_chunk =[]
33                current_token = 0
34
35        if current_chunk:
36            chunks.append(" ".join(current_chunk))
37
38    windows = []
39    window_size = 2
40
41    for i in range(len(chunks) - window_size + 1):
42        window = " ".join(chunks[i:i + window_size])
43        windows.append(window)
44
45    all_chunks = chunks + windows
46
47    return all_chunks
48

```

We have completed the loading and chunking part of our RAG system, now we have to embed and store in vector

Embedding	numeric representation of the text and images, mainly it stored as Float32 format in the vector DB to keep the same meaningful texts or images closer to each other not to a single point of space
-----------	--

- Here is the soft_limit and the Hard_limit I was talking about, this set the limit where to create the chunk.
- If the similarity decrease from the soft_limit then the another chunk will create.
- And the hard_limit is to not make the chunk oversized which will make the chunk's quality decrease.
- And the threshold is to trigger the chunking process, it indicates the similarity in the chunk. If similarity drops means less than the threshold a new chunk will get created.

*all the details is described in the doc string inside the function. It is very important to make a doc string of every function we make, in future system it will make LLM in choosing functions while using langGraph and LangChain.

Now we will store these chunks after embedding them to vector DB:

For embedding I will use SentenceTransformer

```

1 from sentence_transformers import SentenceTransformer
2
3 model = SentenceTransformer('all-mpnet-base-v2')
4 dimention = 384
5

```

```

1 from pathlib import Path
2 import pickle
3 import re
4 import faiss
5 import numpy as np
6 from database.embedder import model
7 from database import load_pdf
8 from database.semantic_chunker import semantic_chunker
9
10 dim = model.get_sentence_embedding_dimension()
11
12 def tokenize(text):
13     """tokenize the texts"""
14
15     return re.findall(r"\b\w+\b", text.lower())
16
17 def ingest_pdf(file_path, doc_id=None):
18     """Add every thing to a folder storing the summaries, the tokens, the chunks.
19     file_path is default rag_db, and the doc_id is none, cause to be added according to the pdf's name"""
20
21     file_path = str(file_path)
22     doc_id = doc_id or Path(file_path).stem
23
24     rag_db_path = Path("rag_db")
25     documents_path = rag_db_path / "documents"
26     rag_db_path.mkdir(exist_ok=True)
27     documents_path.mkdir(parents=True, exist_ok=True)
28
29     text = load_pdf(file_path)
30     sentence = text.split(". ")
31     text_embedding = model.encode(sentence).astype("float32")
32
33     doc_folder = documents_path / doc_id
34     doc_folder.mkdir(parents=True, exist_ok=True)
35
36     chunks = semantic_chunker(sentence, text_embedding)
37     chunk_embeddings = model.encode(chunks).astype("float32")
38     chunk_index = faiss.IndexHNSWFlat(dim, 32)
39     chunk_index.add(chunk_embeddings)
40
41     faiss.write_index(chunk_index, str(doc_folder / "chunk_index.faiss"))
42
43     tokenized_chunks = [tokenize(chunk) for chunk in chunks]
44
45     metadata = {}
46
47     for i, chunk in enumerate(chunks):
48         metadata[i] = {
49             "chunk_id": i,
50             "chunk_text": chunk,
51             "embedding": chunk_embeddings[i],
52         }
53
54     with (doc_folder / "metadata.pkl").open("wb") as metadata_file:
55         pickle.dump(metadata, metadata_file)
56

```

- Now we have transferred the text into vectors and stored in the vector DB.
- We hardcoded it to store the vector DB in a rag_db so that we can easily use that one while retrieving it.
- We used *FAISS (facebook AI similarity search) for semantic search in more efficient way cause it index that organizes your float32 vector, due to which it can search efficiently, in short Indexed Float32.
- All the details like the chunk_id, the chunk_text and the embeddings to a metadata.
- We can convert our model to float16 or respective using Numpy library if the LLM model is not capable of float32 or in any case of fatal.
- In the chunk_index I have set the 'dim' instead of manually setting that cause manually setting the dimension is not robust, may be the data stored is a 768 Or 1025 or 384 or anything so it's the robust method to set dimension.

Now all the requirements are completed, now we have to just query it and see the result.

It's the time to query, our RAG gives the response!

We will convert the query into vector form to do the similarity search in the vector DB

```

1 from sentence_transformers import SentenceTransformer
2 from database.embedder import model
3 model = model
4
5 def embed_query(query):
6     return model.encode([query]).astype('float32')
7

```

We just embedded our query.

So we will route and search in which document our answer is situated at.

```

1 import faiss
2 import pickle
3 from retriever.query_embedder import embed_query
4
5 summary_index = faiss.read_index("rag_db/documents/summary_index.faiss")
6
7 def find_documents(query, top_k=5):
8     query_embedding = embed_query(query)
9     distances, ids = summary_index.search(query_embedding, top_k)
10    return ids[0]
11

```

We have used *FAISS Now we have the chunk retriever to set, so that our rag will give response according to our query:

```

1 import faiss
2 import pickle
3 from retriever.query_embedder import embed_query
4
5 class Chunk_retriever:
6     def __init__(self, doc_id):
7         self.doc_id = doc_id
8         with open(f"rag_db/documents/{doc_id}/metadata.pkl", "rb") as metadata_file:
9             self.metadata = pickle.load(metadata_file)
10
11
12     def retrieve_chunks(self, query, top_k=5):
13
14         query_embedding = embed_query(query)
15         chunk_index = faiss.read_index(f"rag_db/documents/{self.doc_id}/chunk_index.faiss")
16         chunk_distances, chunk_ids = chunk_index.search(query_embedding, top_k)
17
18         results = []
19
20         for idx, chunk_id in enumerate(chunk_ids[0]):
21
22             if chunk_id in self.metadata:
23
24                 results.append({
25                     "chunk_text": self.metadata[chunk_id]["chunk_text"],
26                     "chunk_distance": float(chunk_distances[0][idx])
27                 })
28
29         return results
30

```

Now, it's the last step,

We have to generate the retrieved chunks and send to our LLM for final answer:

```

1 from retriever.document_router import find_documents
2 from retriever.chunk_retriever import chunk_retriever
3
4 retriever = chunk_retriever.retrieve_chunks()
5 def retrieve(query):
6     docs = find_documents(query)
7     context = []
8
9     for doc in docs:
10         doc_id = f"doc_{doc}"
11         chunks = retrieve(query, doc_id)
12         context.extend(chunks)
13
14     final_chunks = context
15
16     return final_chunks

```

Here is our LLM invoking code : (*for now will not use any langChain or langGraph as it's a basic RAG)

```

1  import os
2  import re
3  import ollama
4  from context_builder import build_context
5  from database.ingest_db import ingest_pdf
6  from retriever.chunk_retriever import Chunk_retriever
7
8  class GENERATE:
9      prompt = """
10         Answer the question using the context below. If the answer is not found in the context, say "I don't know".
11         """
12      def __init__(self):
13          self.messages = [{'role': 'system', 'content': self.prompt}]
14
15      def retrieve_context(self, query, doc_id):
16          # text retrieval
17          retriever = Chunk_retriever(doc_id)
18          contexts = retriever.retrieve_chunks(query=query)
19          context_text = build_context(contexts)
20
21          context = f"""
22          TEXT CONTEXT :
23          {context_text}
24          """
25          return context
26

```

We have provided a Static Prompt to our LLM so that it will answer like that. (* in upcoming session while we use langChain we will learn how to set dynamic prompt according to the user)

We can also add context builder to set the output in a specific level to prevent your GPU from getting bottlenecked .
If we will set the retrieved answer between certain level then we will add a retriever_context.

Now let's generate our answer and invoke our LLM:

```

27  def generate(self, user_input:str,doc_id):
28
29      context = self.retrieve_context(user_input,doc_id)
30
31      question_prompt = f"""
32      Context:
33      {context}
34      Question:
35      {user_input}
36      """
37      self.messages = [
38          {"role": "system", "content": self.prompt},
39          {"role": "user", "content": question_prompt},
40      ]
41
42      response = ollama.chat(model='llama3.1:8b-instruct-q5_K_S', messages=self.messages, stream=True, options={"num_thread":8}, keep_alive="1m")
43
44      for chunk in response:
45          content = chunk.get("message", {}).get("content", "")
46          if not content:
47              continue
48
49          final_text = re.sub(r'\[.*?\]', '', content)
50          if final_text:
51              yield final_text
52

```

Here we have used our Ollama model, you can download the model from the Ollama website try to download between 8 billion parameter model otherwise your system will crash, cause it can't handle such large model (*if you have a good GPU spec then can consider 10 or 12 billion parameter model) or can also use any LLM API (Claude, Openai, Gemini, grok, etc.)

You can see in the .chat we have set a parameter called stream =True cause this will feed data to the model in a sequence instead of at ones.

This will give in a nice gpt like response.

We have set a memory to our LLM so that it will remember the conversation in the chat.

We will create a CLI model where we just loop our call with a break point being set for exit from the loop:

```

53 def main():
54     rag = GENERATE()
55
56     files=[]
57     if not os.path.exists("rag_db"):
58         os.makedirs("rag_db", exist_ok=True)
59
60     doc_id = None
61
62
63     user = input("\nWant to add file?(yes/no): ").strip()
64
65     if "yes" in user:
66         pdf = input("\nEnter PDF file with .pdf extension: ").strip()
67         if pdf.endswith(".pdf"):
68             doc_id = os.path.splitext(pdf)[0]
69             ingest_pdf(pdf, doc_id)
70             print(f"[+] PDF {pdf} indexed successfully.")
71         else:
72             print("[-] Invalid PDF file.")
73     elif "no" in user:
74         if not os.listdir("rag_db/documents"):
75             print("[-] No documents found in dataset")
76         doc_id = input("\nEnter the document id to use: ")
77         if not doc_id:
78             doc_id = input("enter valid one: ")
79
80     while True:
81         try:
82             text = input("\nEnter your question(else 'bye' to exit): ").strip()
83
84             if "bye" in text:
85                 break
86             print("\nResearcher: ", end="", flush=True)
87             response_started = False
88
89             for token in rag.generate(text, doc_id):
90                 response_started = True
91                 print(token, end="", flush=True)
92
93             if response_started:
94                 print()
95             else:
96                 print("No response generated.")
97
98         except Exception as e:
99             print(f"Error: {e}")
100
101 if __name__ == "__main__":
102     main()

```

So, for LLM we will use Ollama's model : llama3.1:8b-instruct-q5_K_S, so this means it is a 8 billion llama model which is 5-bit (run on float16) quantized model and the K_S is the model size compression S- aggressive compression M- balanced choice and K is for K-quant a specific llama.cpp format that stores the model in a GGUF (gpt-generated unified format) format which is the successor of GGML(gpt-generated model language).

You can also set the model to run on CPU if you either don't have a GPU or less powered one.

You have to just change the ollama model calling function with a specific parameter::

This will force fully change the model to use CPU as we set the GPU as '0' so we have to set the number of thread the CPU should use in this case we will play safe, so you should put it in between 10 to 5 based on your CPU's total threads.

```

response = ollama.chat(model='llama3.1:8b-instruct-q5_K_S', messages=self.messages, stream=True,
options={"num_thread":8,"num_gpu":0,"keep_alive":"1m"})

```

*make sure that you have running ollama in background (the ollama app, it's the area where the model get connects and able to run)

So far now we have success fully learned how to make a simple RAG system using vector DB and our LLM running on GPU or CPU.

In our next session we will be learning different kind of RAG system we can use according to different use cases. And at the end you will get all the ideas to build a production based RAG system.

We will see you there!!

Thanks for reading this document, feel free to contact if something is wrongly or not properly described.

@metamorphtech.uk@gmail.com

GitHub : <https://github.com/swayansu951>